Documento de Evidências

Laboratório de Containers com Docker — Da aplicação C# ao deploy versionado

Integração DevOps · Unidade 03 · Prof. Daniel Lim-Apo · Ciência da Computação

Aluno(s): _____

Matrícula: _____

Data: _____ / _____ / 2026

Como usar este documento. Os arquivos (itens 2 a 4) já estão transcritos. Os itens de execução (5 a 10) exigem Docker e .NET na sua máquina: rode `bash run-lab.sh` na pasta DevOpsWebApi (ou os comandos indicados), tire o print de cada passo e cole nos quadros vermelhos. Os campos em branco não foram preenchidos de propósito: a evidência deve ser a saída real da sua execução.

#	Evidência obrigatória	Situação
1	URL do repositório Git	Colar print/saída
2	Dockerfile	Arquivo transcrito
3	.dockerignore	Arquivo transcrito
4	compose.yaml	Arquivo transcrito
5	Imagens 1.0.0 e 1.1.0	Colar print/saída
6	Container de staging	Colar print/saída
7	Container de production	Colar print/saída
8	Health check bem-sucedido	Colar print/saída
9	Versão 1.2.0 com falha	Colar print/saída
10	Rollback para 1.1.0	Colar print/saída
11	README com os principais comandos	Arquivo transcrito

1. URL do repositório Git

Publique a pasta DevOpsWebApi (com Dockerfile, .dockerignore, compose.yaml, README.md, Program.cs e versions/) em um repositório e informe a URL:

URL: https://github.com/_____/_____

```
git init
git add .
git commit -m "Laboratorio Docker"
git remote add origin <URL>
git push -u origin main
```

COLE AQUI: print do repositório no GitHub

2. Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY DevOpsWebApi.csproj .
RUN dotnet restore

COPY . .
RUN dotnet publish \
    -c Release \
    -o /app/publish \
    --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app

COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "DevOpsWebApi.dll"]
```

Build em dois estágios: o estágio *build* usa o SDK do .NET 10 para restaurar e publicar; o estágio *runtime* usa apenas a imagem ASP.NET e recebe só os arquivos publicados, resultando em imagem final menor. O restore é feito antes da cópia do código para aproveitar o cache de camadas.

3. .dockerignore

```
bin/
obj/
.git/
.github/
.vscode/
*.user
*.suo
```

Evita enviar bin/, obj/, histórico Git e arquivos de IDE ao contexto de build.

4. compose.yaml

```
services:
  staging:
    image: devops-webapi:1.1.0
    container_name: devops-api-staging
    ports:
      - "8081:8080"
    environment:
      APP_ENVIRONMENT: staging
      APP_VERSION: "1.1.0"

  production:
    image: devops-webapi:1.1.0
    container_name: devops-api-prod
    ports:
      - "8082:8080"
    environment:
      APP_ENVIRONMENT: production
      APP_VERSION: "1.1.0"
```

Declara staging (porta 8081) e production (porta 8082) a partir da mesma imagem 1.1.0, mudando apenas as variáveis de ambiente.

5. Evidência das imagens 1.0.0 e 1.1.0

Comando(s):

```
docker build -t devops-webapi:1.0.0 .
docker build -t devops-webapi:1.1.0 .
docker images devops-webapi
```

Esperado: duas linhas, devops-webapi 1.1.0 e devops-webapi 1.0.0, coexistindo no host.

COLE AQUI: saída do docker images mostrando as tags 1.0.0 e 1.1.0

6. Evidência do container de staging

Comando(s):

```
docker run -d --name devops-api-staging -p 8081:8080 \
  -e APP_ENVIRONMENT=staging -e APP_VERSION=1.1.0 devops-webapi:1.1.0
docker ps
curl http://localhost:8081/
```

COLE AQUI: docker ps + resposta de http://localhost:8081/ (environment = staging)

7. Evidência do container de production

Comando(s):

```
docker run -d --name devops-api-prod -p 8082:8080 \
  -e APP_ENVIRONMENT=production -e APP_VERSION=1.1.0 devops-webapi:1.1.0
docker ps
curl http://localhost:8082/
```

COLE AQUI: docker ps + resposta de http://localhost:8082/ (environment = production)

8. Resultado de um health check bem-sucedido

Comando(s):

```
curl -f http://localhost:8082/health      # Windows: curl.exe
```

COLE AQUI: resposta JSON com "status":"healthy" e código de saída 0

9. Evidência da versão 1.2.0 com falha

Comando(s):

```
# Program.cs com /health retornando HTTP 500 (versions/Program.1.2.0.cs)
docker build -t devops-webapi:1.2.0 .
docker rm -f devops-api-staging
docker run -d --name devops-api-staging -p 8081:8080 \
  -e APP_ENVIRONMENT=staging -e APP_VERSION=1.2.0 devops-webapi:1.2.0
curl -f -i http://localhost:8081/health
```

Decisão: o health check falhou em staging, portanto a 1.2.0 não deve ser promovida.

COLE AQUI: saída do curl indicando falha HTTP 500 (curl: (22) ... error: 500)

COLE AQUI: (opcional) docker ps mostrando 1.2.0 no ambiente em que falhou

10. Evidência do rollback para 1.1.0

Comando(s):

```
docker rm -f devops-api-prod
docker run -d --name devops-api-prod -p 8082:8080 \
  -e APP_ENVIRONMENT=production -e APP_VERSION=1.1.0 devops-webapi:1.1.0
curl -f http://localhost:8082/health
```

COLE AQUI: produção com 1.2.0 falhando (antes do rollback)

COLE AQUI: produção após rollback: health OK com "version":"1.1.0"

11. README

```
# Laboratório Docker

API ASP.NET Core (.NET 10) containerizada com Docker, versionada por tags e implantada em staging/production.

## Construção
docker build -t devops-webapi:1.0.0 .

## Execução
docker run -d --name devops-api -p 8080:8080 devops-webapi:1.0.0

## Staging
docker run -d --name devops-api-staging -p 8081:8080 -e APP_ENVIRONMENT=staging -e APP_VERSION=1.0.0 devops-
webapi:1.0.0

## Production
docker run -d --name devops-api-prod -p 8082:8080 -e APP_ENVIRONMENT=production -e APP_VERSION=1.0.0 devops-
webapi:1.0.0

## Health Check
curl -f http://localhost:8081/health   (Windows: curl.exe)

## Nova versão
docker build -t devops-webapi:1.1.0 .

## Falha controlada
docker build -t devops-webapi:1.2.0 .   # /health retorna HTTP 500 de propósito
curl -f http://localhost:8081/health    # falha: curl retorna erro 22

## Rollback
docker rm -f devops-api-prod
docker run -d --name devops-api-prod -p 8082:8080 -e APP_ENVIRONMENT=production -e APP_VERSION=1.1.0 devops-
webapi:1.1.0

## Docker Compose
docker compose up -d
docker compose ps
docker compose down

## Execução automatizada
`versions/` guarda o Program.cs de cada release (1.0.0, 1.1.0, 1.2.0).
`run-lab.sh` (Linux/macOS/Git Bash) executa todo o laboratório e grava as saídas em `evidencias/log.txt`.
```

Anexo — Perguntas da defesa (respostas de apoio)

Imagem × container?

Imagem é o modelo imutável (filesystem, runtime, app, metadados); container é uma instância em execução criada a partir dela. Uma imagem gera vários containers.

Função do Dockerfile?

Descreve passo a passo como construir a imagem, tornando o build reproduzível.

O que significa 8081:8080?

porta-host:porta-container. Acessos em localhost:8081 são encaminhados à porta 8080 dentro do container.

Por que variáveis de ambiente?

Permitem usar a mesma imagem em ambientes diferentes, mudando só a configuração, sem recompilar.

Vantagem do multi-stage?

SDK, compiladores e código-fonte ficam só no estágio de build; a imagem final contém apenas runtime + app publicado (menor e com menos superfície).

Por que tags 1.0.0 e 1.1.0?

Identificação, rastreabilidade, reprodução e possibilidade de rollback; evita a ambiguidade do latest.

Por que staging primeiro?

Para validar a release (inicialização, endpoints, health) antes de expô-la como produção.

Como o health check detectou a falha?

O /health da 1.2.0 devolve HTTP 500; o curl -f trata status ≥ 400 como erro (código de saída 22).

Como foi o rollback?

Removeu-se o container de produção com a 1.2.0 e subiu-se outro com a imagem 1.1.0, que continuava disponível no host.

Como o GitHub Actions automatizaria?

Um workflow disparado por push executaria build, testes, docker build com tag da versão, deploy em staging, health check como quality gate e promoção para produção só se passar.

Pergunta de fixação: interromper a promoção e investigar a causa. A informação que sustenta a decisão é o HTTP 500 no /health da 1.2.0 em staging, o quality gate falhou.